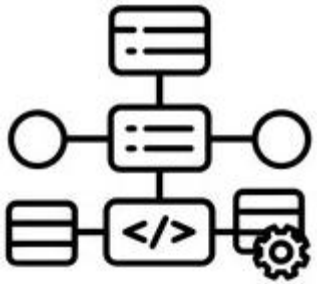
Architectural Styles and Patterns

Estilos y Patrones Arquitectonicos

Universidad de La Sabana
FACULTAD DE INGENIERÍA

Universidad de La Sabana
FACULTAD DE INGENIERIA

Agenda



Architectural Styles and Patterns

- Why
  - Software Development Lifecycle
  - Business Driver/Motivator
- What/What Extend
  - Software Requirements
    - Gathering — QAW
    - ASR
  - Quality Attributes
    - Availability
    - Performance
    - .....
  - Tactics
    - Heartbeat
    - Voting
    - Retry
    - Increase Resources
    - .....
  - Architectural Patterns
    - Rate Limiting
    - Sidecar
    - CQRS
    - .....
  - Architectural Styles
    - Layered
    - Modular Monolith
    - EDA
    - Microservices
    - .....
- How
  - Attribute-Driven Design - ADD

SDLC and SW Req

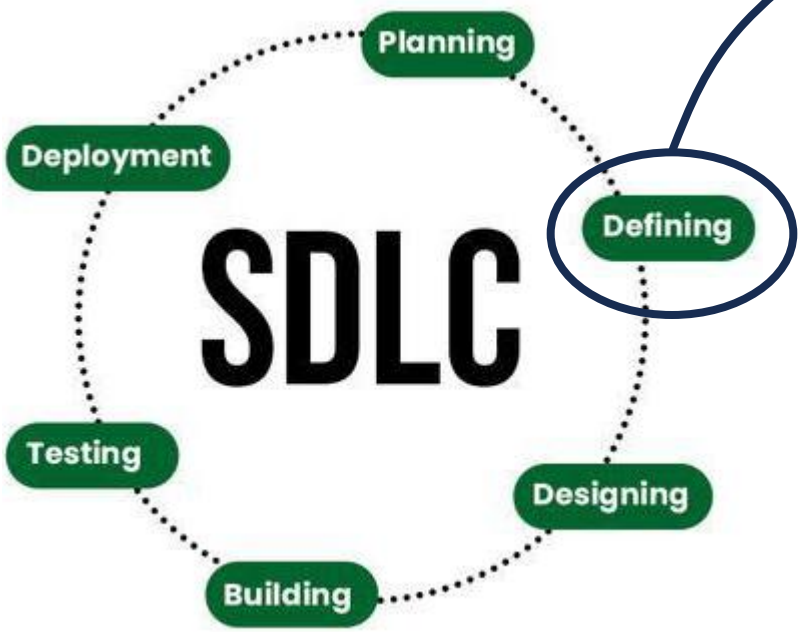


TABLE 1-1 Some types of requirements information

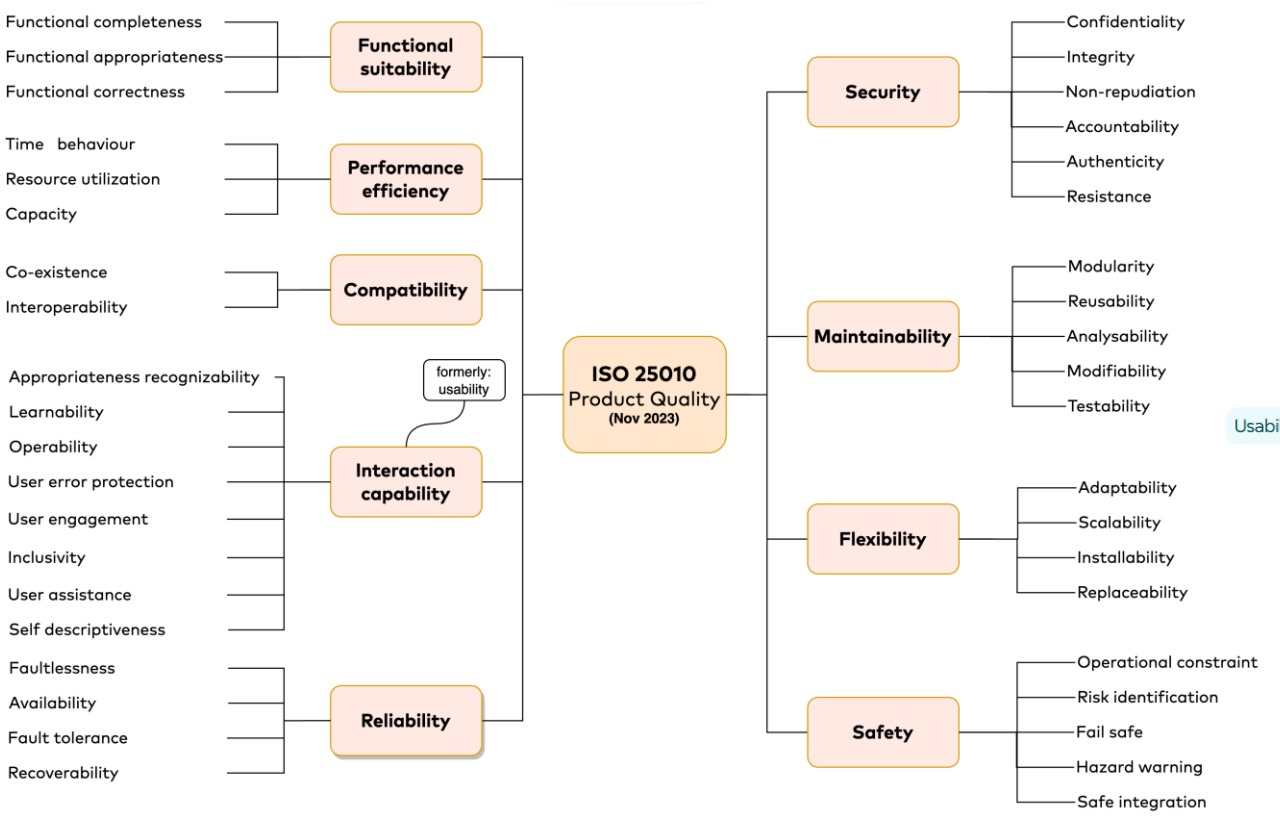
Term	Definition
Business requirement	A high-level business objective of the organization that builds a product or of a customer who procures it.
Business rule	A policy, guideline, standard, or regulation that defines or constrains some aspect of the business. Not a software requirement in itself, but the origin of several types of software requirements.
Constraint	A restriction that is imposed on the choices available to the developer for the design and construction of a product.
External interface requirement	A description of a connection between a software system and a user, another software system, or a hardware device.
Feature	One or more logically related system capabilities that provide value to a user and are described by a set of functional requirements.
Functional requirement	A description of a behavior that a system will exhibit under specific conditions.
Nonfunctional requirement	A description of a property or characteristic that a system must exhibit or a constraint that it must respect.
Quality attribute	A kind of nonfunctional requirement that describes a service or performance characteristic of a product.
System requirement	A top-level requirement for a product that contains multiple subsystems, which could be all software or software and hardware.
User requirement	A goal or task that specific classes of users must be able to perform with a system, or a desired product attribute.

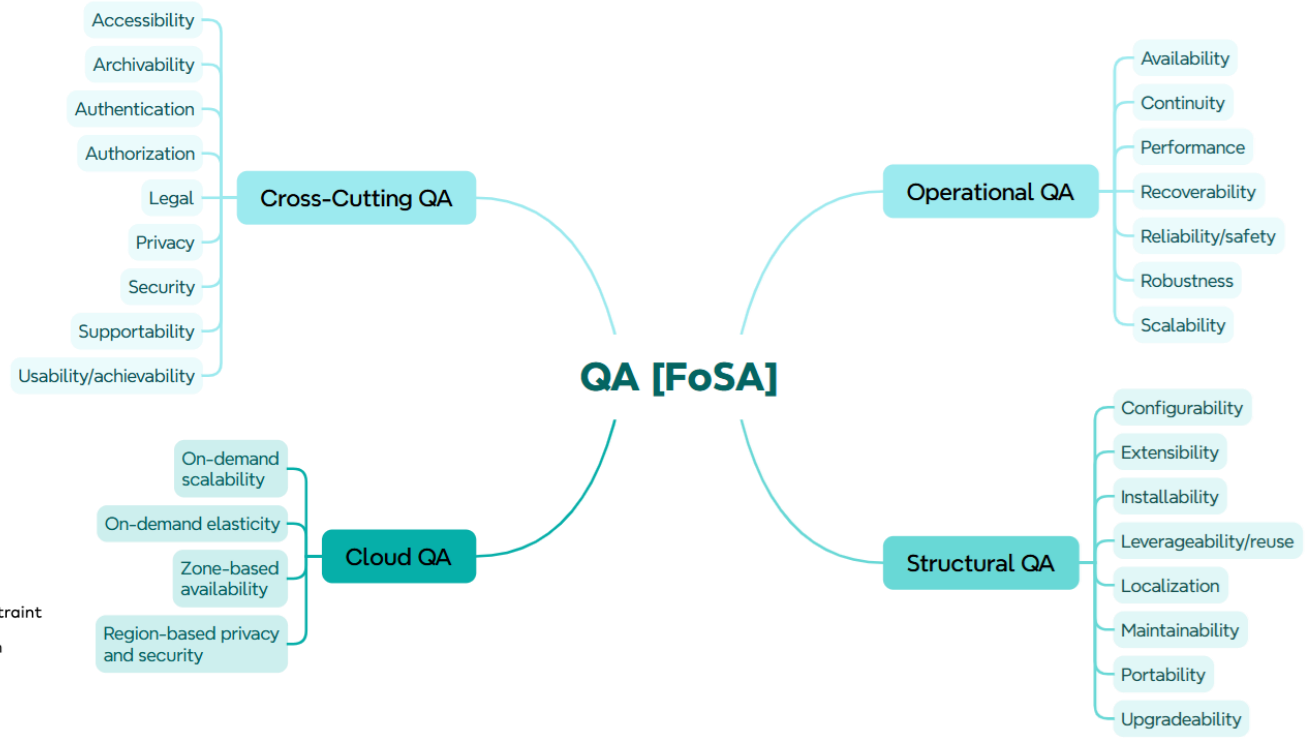
There are a different names for the same property: {System} Quality Attribute [SAiP, SWRe], Architectural Characteristics [FoSA], Capabilities (of the System) [HFSoA], “-illities”, cross-cutting requirements, capabilities….

[SWRe]

Universidad de La Sabana
FACULTAD DE INGENIERIA

Arch Characteristics/Quality attributes





Even thought those sources and classification, there are ambiguous definition, i.e. *learnability* (users or system?) or for elements, TCP/IP (*available vs reliable*) [SoFA].

[SAiP], [FoSA] or link

Quality Attributes and its scope

Table 14-5. Translating quality attributes into technical specifications

Quality attributes	Likely technical information category
Installability, integrity, interoperability, reliability, robustness, safety, security, usability, verifiability	Functional requirement
Availability, efficiency, modifiability, performance, reliability, scalability	System architecture
Interoperability, security, usability	Design constraint
Efficiency, modifiability, portability, reliability, reusability, scalability, verifiability, usability	Design guideline
Portability	Implementation constraint

Over-engineering !

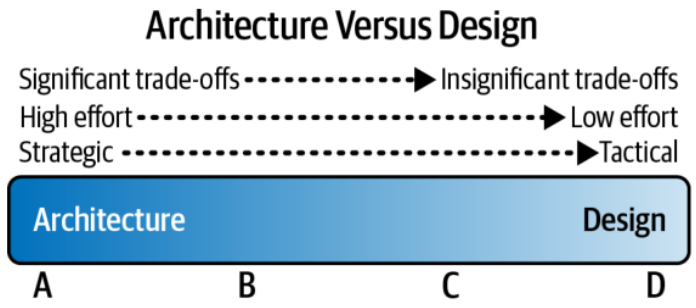


Figure 2-1. The spectrum between architecture and design

TABLE 14-1 Some software quality attributes

External quality	Brief description
Availability	The extent to which the system's services are available when and where they are needed
Installability	How easy it is to correctly install, uninstall, and reinstall the application
Integrity	The extent to which the system protects against data inaccuracy and loss
Interoperability	How easily the system can interconnect and exchange data with other systems or components
Performance	How quickly and predictably the system responds to user inputs or other events
Reliability	How long the system runs before experiencing a failure
Robustness	How well the system responds to unexpected operating conditions
Safety	How well the system protects against injury or damage
Security	How well the system protects against unauthorized access to the application and its data
Usability	How easy it is for people to learn, remember, and use the system
Internal quality	**Brief description**
Efficiency	How efficiently the system uses computer resources
Modifiability	How easy it is to maintain, change, enhance, and restructure the system
Portability	How easily the system can be made to work in other operating environments
Reusability	To what extent components can be used in other systems
Scalability	How easily the system can grow to handle more users, transactions, servers, or other extensions
Verifiability	How readily developers and testers can confirm that the software was implemented correctly

TIP

Never strive for the *best* architecture; aim for the *least worst* architecture.

[FoSA] and [SWRe]

Explicit or Implicit Definition

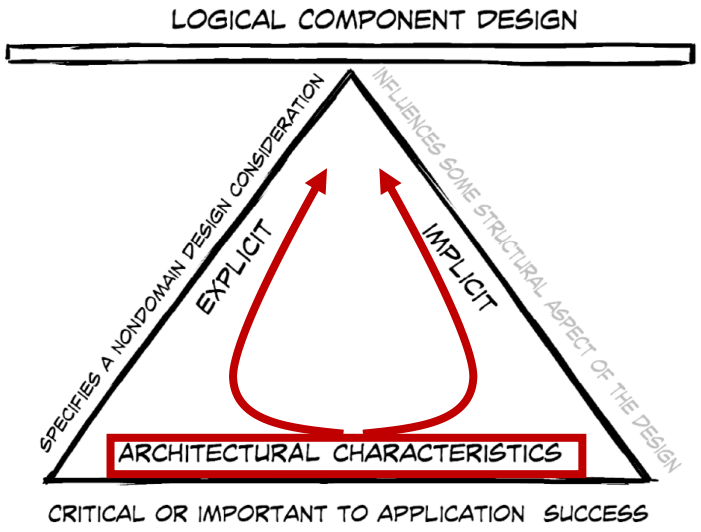


Table 5-1. Translating domain concerns into architectural characteristics

Domain concern	Architectural characteristics
Mergers and acquisitions	Interoperability, scalability, adaptability, extensibility
Time to market	Agility, testability, deployability
User satisfaction	Performance, availability, fault tolerance, testability, deployability, agility, security
Competitive advantage	Agility, testability, deployability, scalability, availability, fault tolerance
Time and budget	Simplicity, feasibility

Low-Code or No-Code Platforms

Over-engineering !

What are the priorities ?

There are no wrong answers in architecture, only expensive ones.

—One of Mark's famous quotes

Universidad de La Sabana
FACULTAD DE INGENIERIA

Prioritize attributes

Synergetic

	Availability	Efficiency	Installability	Integrity	Interoperability	Modifiability	Performance	Portability	Reliability	Reusability	Robustness	Safety	Scalability	Security	Usability	Verifiability
Availability									+		+					
Efficiency	+				−	−	+	−			−		+		−	
Installability	+								+					+		
Integrity			−		−		−			−		+		+	−	−
Interoperability	+		−	−			−	+	+		+	−		−		
Modifiability	+		−				−		+	+			+			+
Performance		+			−	−		−			−		−		−	
Portability		−			+	−	−			+				−	−	+
Reliability	+	−		+		+	−				+	+		+	+	+
Reusability		−		−	+	+	−	+						−		+
Robustness	+	−	+	+	+		−		+			+	+	+	+	
Safety		−		+	+		−				+			+	−	−
Scalability	+	+		+			+	+	+		+					
Security	+			+	+		−	−	+		+	+			−	−
Usability		−	+				−	−	+		+	+				−
Verifiability	+		+	+		+			+	+	+	+		+	+	

Figure 14-2. Positive and negative relationships among selected quality attributes.

Architecture characteristics worksheet

System/project: _____

Architect/team: _____

Top 3	Driving characteristics	Implicit characteristics
☐		Feasibility (cost/time)
☐		Security
☐		Maintainability
☐		Observability
☐		
☐		Others considered
☐		

Figure 5-3. Architectural characteristics worksheet

Step 1: Start with a broad taxonomy
Step 2: Reduce the list
Step 3: Prioritize the attributes
Step 4: Elicit specific expectations for each attribute
Step 5: Specify well-structured quality requirements

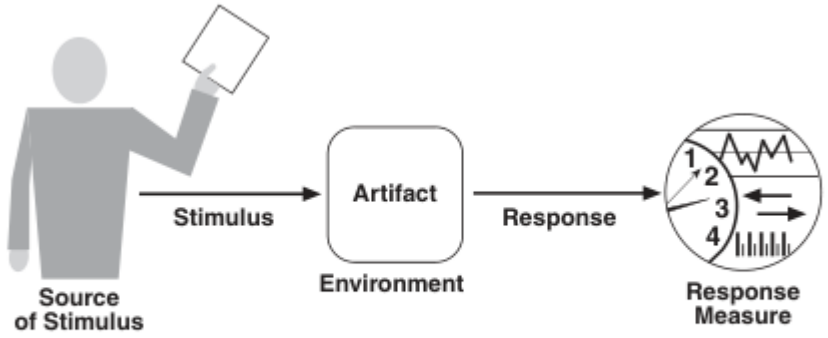
Attribute	Score	Availability	Integrity	Performance	Reliability	Robustness	Security	Usability	Verifiability
Availability	2		^	^	^	<	^	^	<
Integrity	6			<	<	<	^	<	<
Performance	4				<	<	^	^	<
Reliability	2					<	^	^	^
Robustness	1						^	^	<
Security	7							<	<
Usability	5								<
Verifiability	1								

Figure 14-1. Sample quality attribute prioritization for an airport check-in kiosk.

[SAIP], [FoSA] [SWRe]

- HLD
- Effort

Who needs an architect – Fowler, who is architect vs Design: Link



IGURE 4.1 The parts of a quality attribute scenario

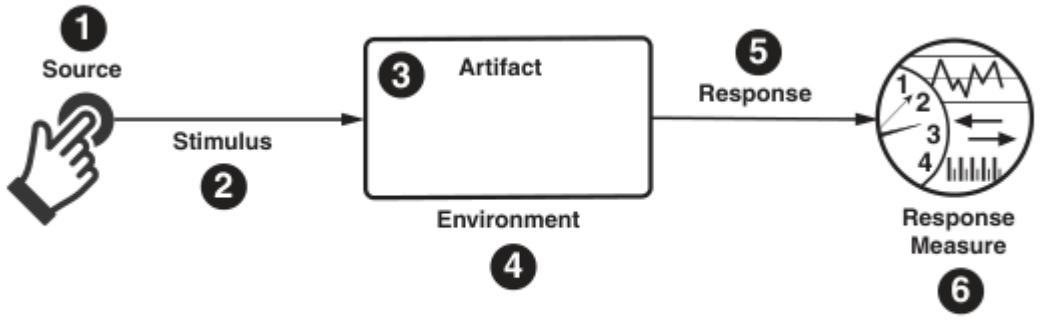
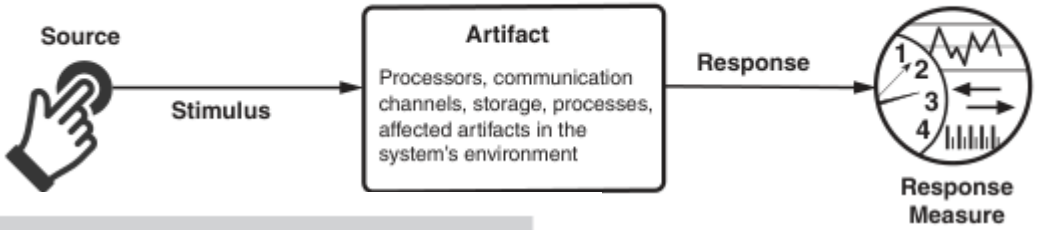
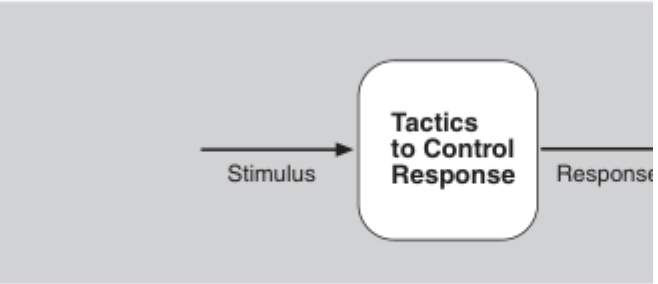


FIGURE 3.1 The parts of a quality attribute scenario



Prevent the fault from becoming a failure	Time or time interval when the system must be available
Detect the fault	Availability percentage
Recover from the fault	Time to detect the fault



IGURE 4.3 Tactics are intended to control responses to

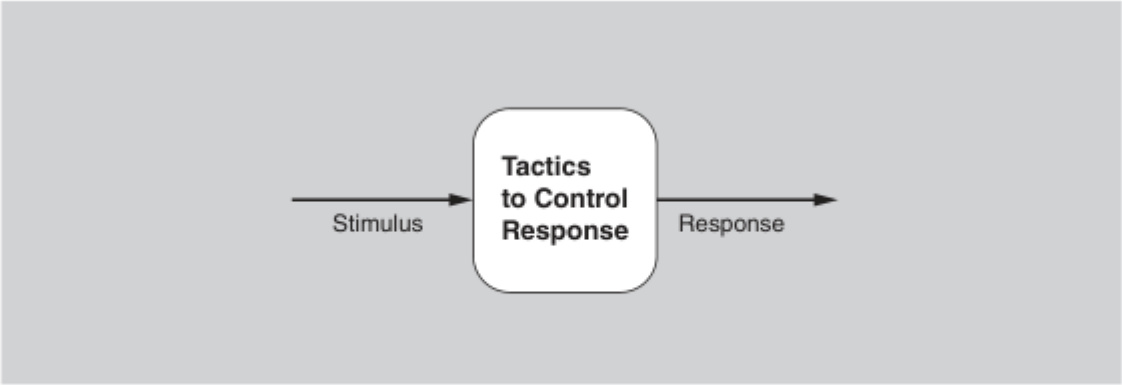


FIGURE 3.3 Tactics are intended to control responses to stimuli.

Universidad de La Sabana
FACULTAD DE INGENIERIA

- 3rd Edicion

PART TWO: QUALITY ATTRIBUTES
- 4 Understanding Quality Attributes
- 5 Availability
- 6 Interoperability
- 7 Modifiability
- 8 Performance
- 9 Security
- 10 Testability
- 11 Usability
- 12 Other Quality attributes
- 13 Architectural Tactics and Patterns
- 14 Quality Attribute Modeling and Analysis

- 4th Edition

PART II: QUALITY ATTRIBUTES
- CHAPTER 3 Understanding Quality Attributes
- CHAPTER 4 Availability
- CHAPTER 5 Deployability
- CHAPTER 6 Energy Efficiency
- CHAPTER 7 Integrability
- CHAPTER 8 Modifiability
- CHAPTER 9 Performance
- CHAPTER 10 Safety
- CHAPTER 11 Security
- CHAPTER 12 Testability
- CHAPTER 13 Usability
- CHAPTER 14 Working with Other Quality Attributes

- Availability
- Modifibility
- Performance
- Security
- Testability
- Usability

Universidad de La Sabana
FACULTAD DE INGENIERIA

3rd Edition

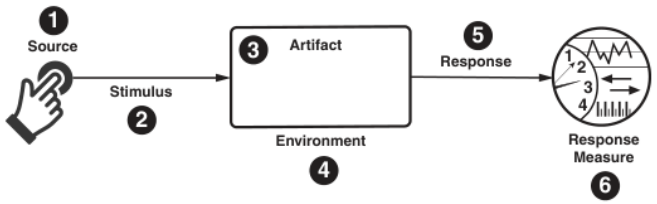


FIGURE 3.1 The parts of a quality attribute scenario

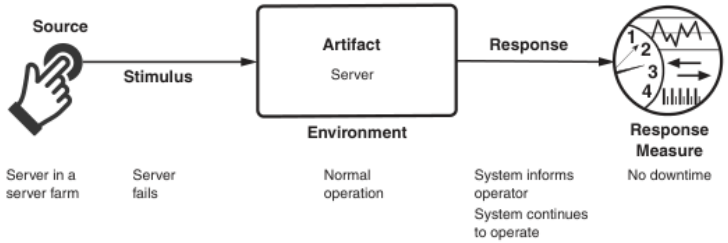


FIGURE 4.1 Sample concrete availability scenario

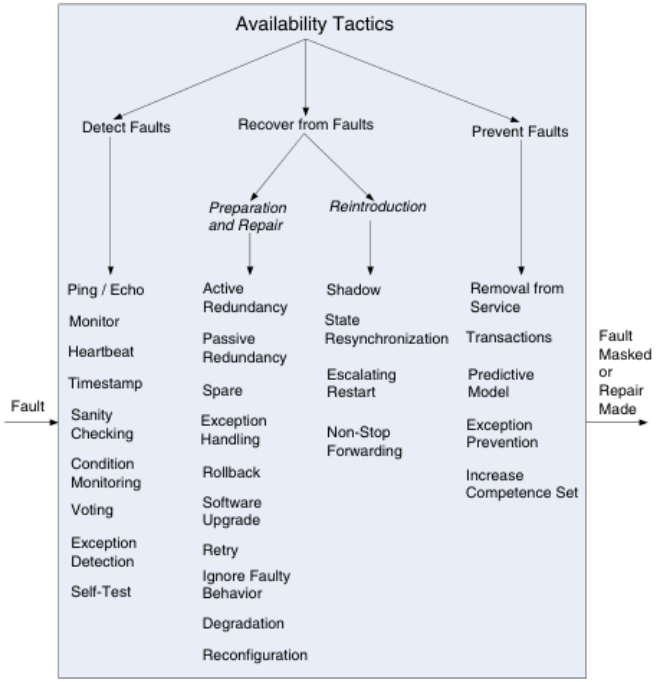


FIGURE 5.5 Availability tactics

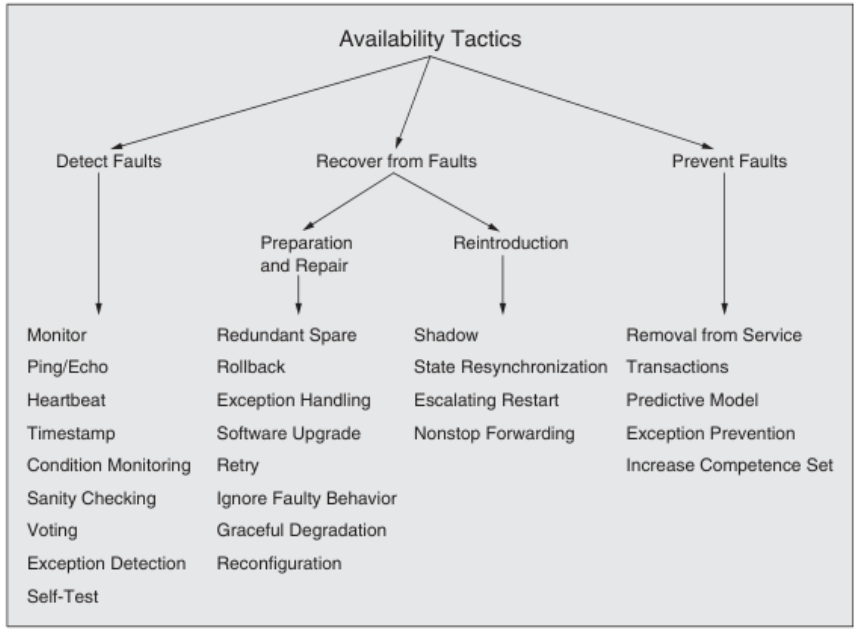


FIGURE 4.3 Availability tactics

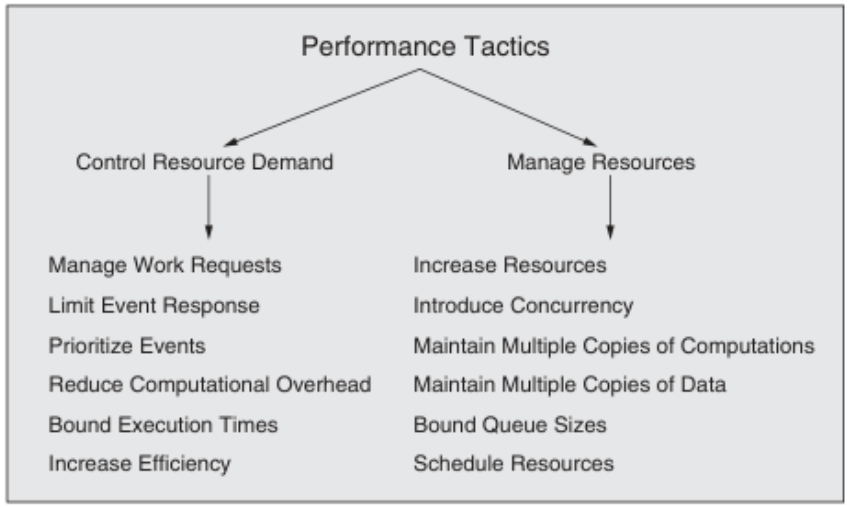
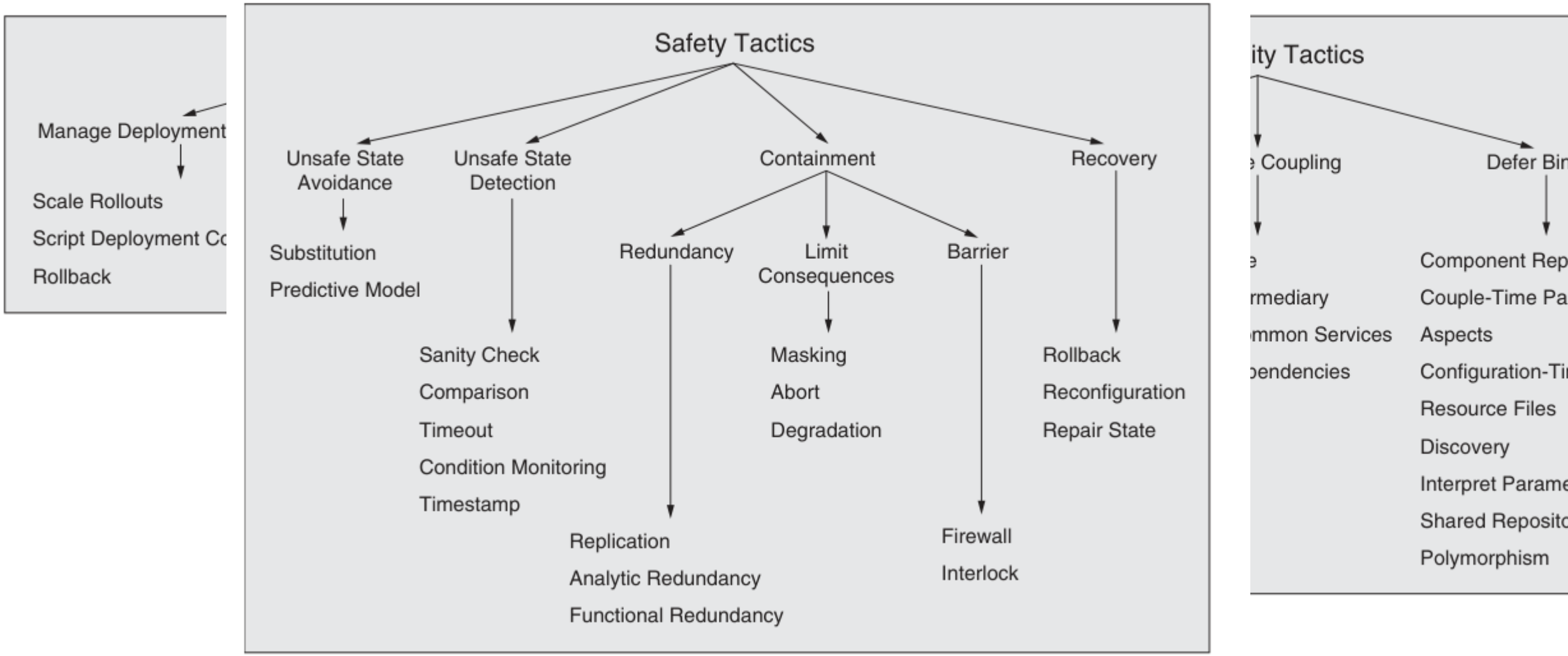
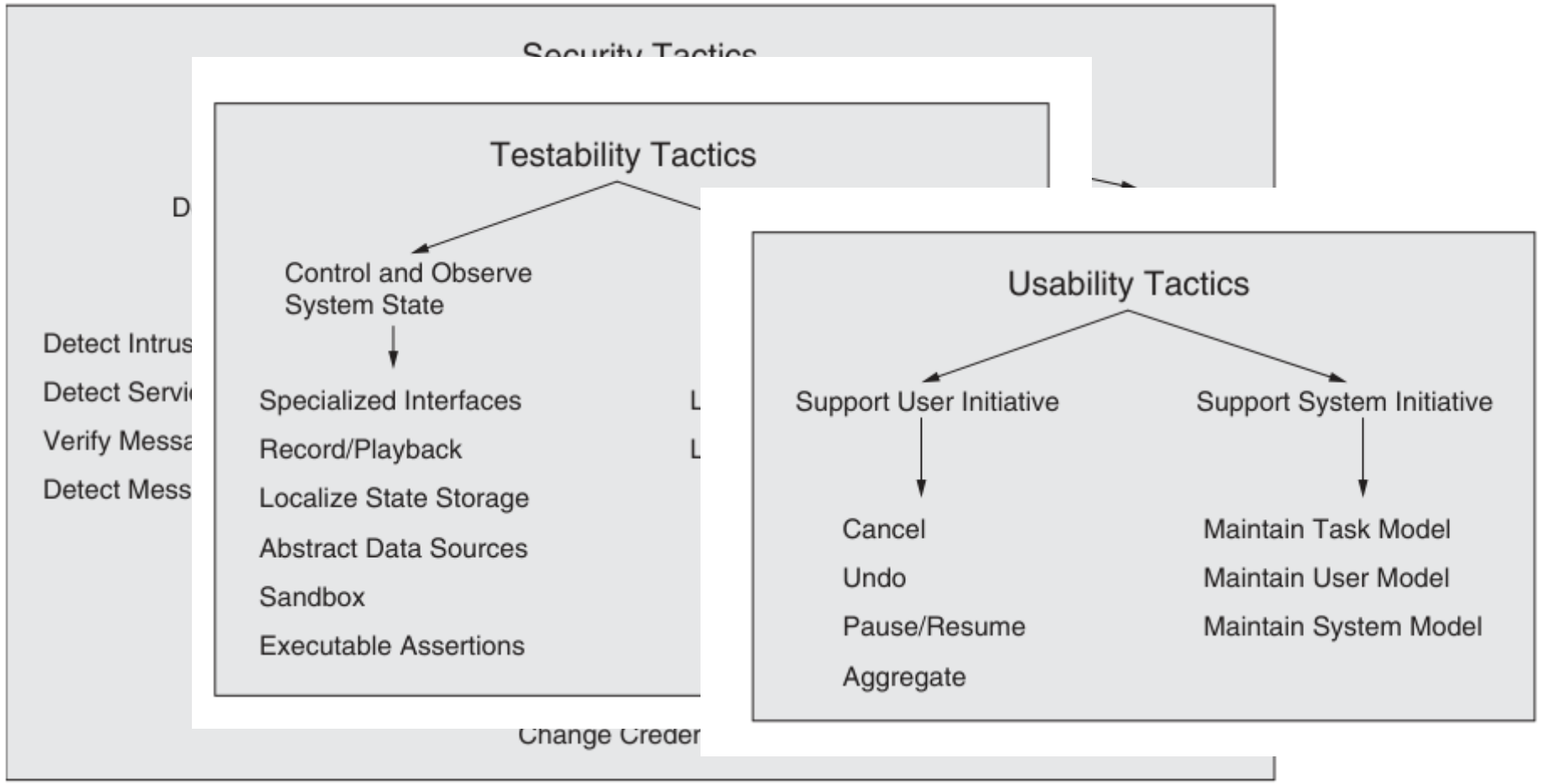


FIGURE 9.3 Performance tactics


Manage Deployment
Scale Rollouts
Script Deployment Co
Rollback
Safety Tactics
Unsafe State Avoidance
Unsafe State Detection
Containment
Recovery
Substitution
Predictive Model
Redundancy
Limit Consequences
Barrier
Sanity Check
Comparison
Timeout
Condition Monitoring
Timestamp
Masking
Abort
Degradation
Rollback
Reconfiguration
Repair State
Replication
Analytic Redundancy
Functional Redundancy
Firewall
Interlock
ity Tactics
e Coupling
Defer Bin
e
rmediary
mmon Services
pendencies
Component Rep
Couple-Time Pa
Aspects
Configuration-Ti
Resource Files
Discovery
Interpret Parame
Shared Reposito
Polymorphism


Testability Tactics
Control and Observe System State
Specialized Interfaces
Record/Playback
Localize State Storage
Abstract Data Sources
Sandbox
Executable Assertions
Usability Tactics
Support User Initiative
Support System Initiative
Cancel
Undo
Pause/Resume
Aggregate
Maintain Task Model
Maintain User Model
Maintain System Model
Detect Intrus
Detect Servi
Verify Messa
Detect Mess

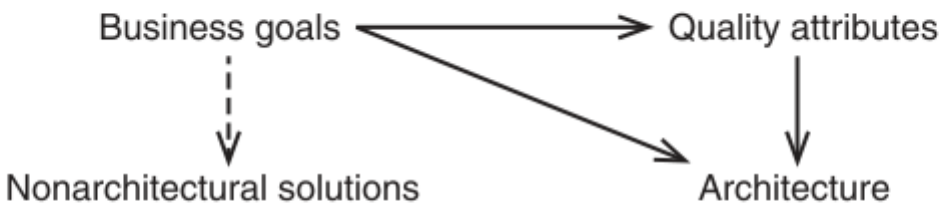


FIGURE 19.1 Some business goals may lead to quality attribute requirements, or lead directly to architectural decisions, or lead to non-architectural solutions.

TABLE 19.1 Tabular Form of the Utility Tree for a System in the Healthcare Space

Quality Attribute	Attribute Refinement	ASR Scenario
Performance	Transaction response time	A user updates a patient's account in response to a change-of-address notification while the system is under peak load, and the transaction completes in less than 0.75 seconds. (H, H)
	Throughput	At peak load, the system is able to complete 150 normalized transactions per second. (M, M)
Usability	Proficiency training	A new hire with two or more years' experience in the business can learn, with 1 week of training, to execute any of the system's core functions in less than 5 seconds. (M, L)
	Efficiency of operations	A hospital payment officer initiates a payment plan for a patient while interacting with that patient and completes the process with no input errors. (M, M)
Configurability	Data configurability	A hospital increases the fee for a particular service. The configuration team makes and tests the change in 1 working day; no source code needs to change. (H, L)
Maintainability	Routine changes	A maintainer encounters response-time deficiencies, fixes the bug, and distributes the bug fix with no more than 3 person-days of effort. (H, M)
		A reporting requirement requires a change to the report-generating metadata. Change is made and tested in 4 person-hours of effort (M, L)
	Upgrades to commercial components	The database vendor releases a new major version that is successfully tested and installed in less than 3 person-weeks. (H, M)
	Adding new feature	A feature that tracks blood bank donors is created and successfully integrated within 2 person-months. (M, M)
Security	Confidentiality	A physical therapist is allowed to see that part of a patient's record dealing with orthopedic treatment, but not other parts or any financial information. (H, M)
	Resisting attacks	The system repels an unauthorized intrusion attempt and reports the attempt to authorities within 90 seconds. (H, M)
Availability	No down time	The database vendor releases new software, which is hot-swapped into place, with no downtime. (H, L)
		The system supports 24/7/365 web-based account access by patients. (M, M)

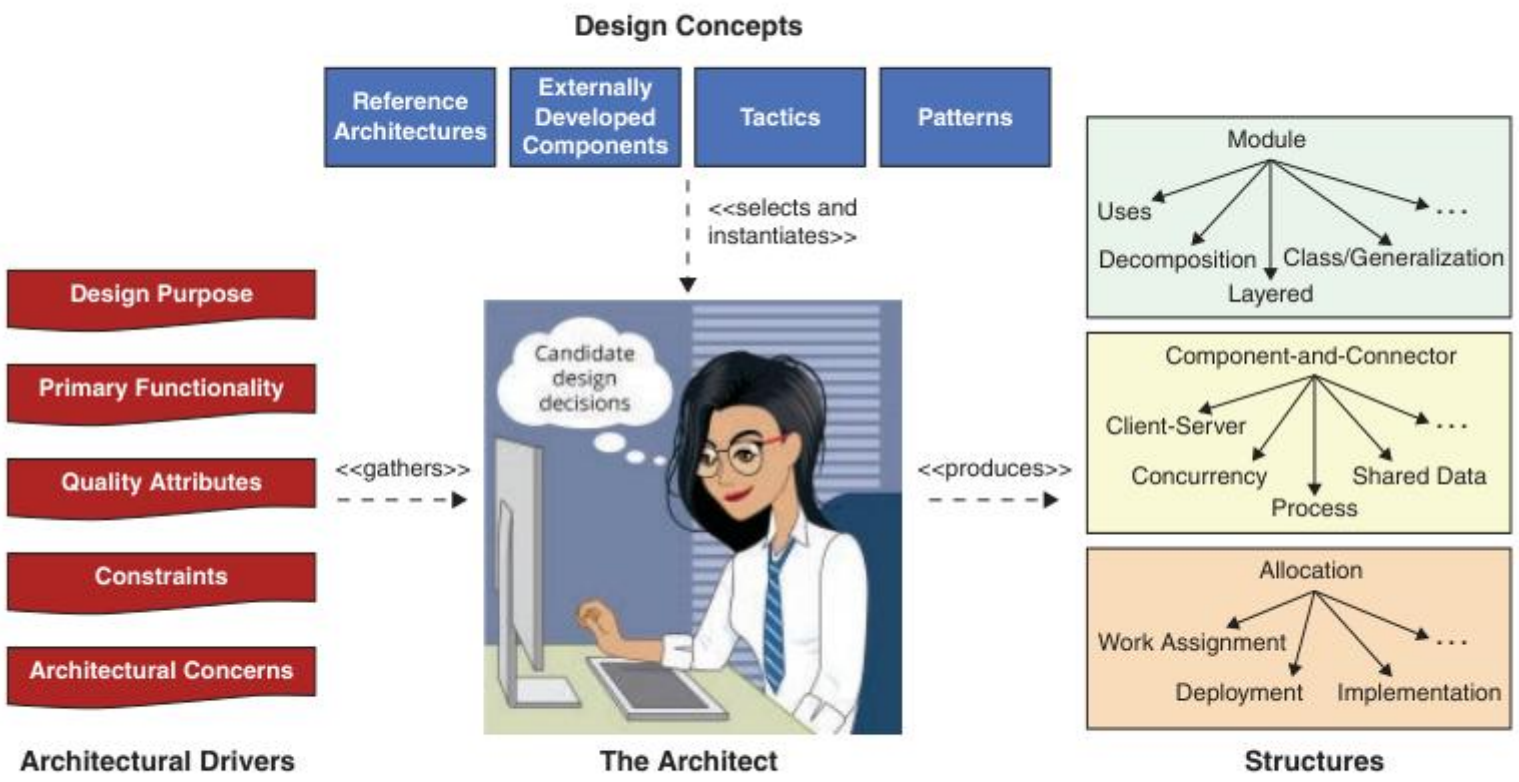


FIGURE 20.1 Overview of the architecture design activity

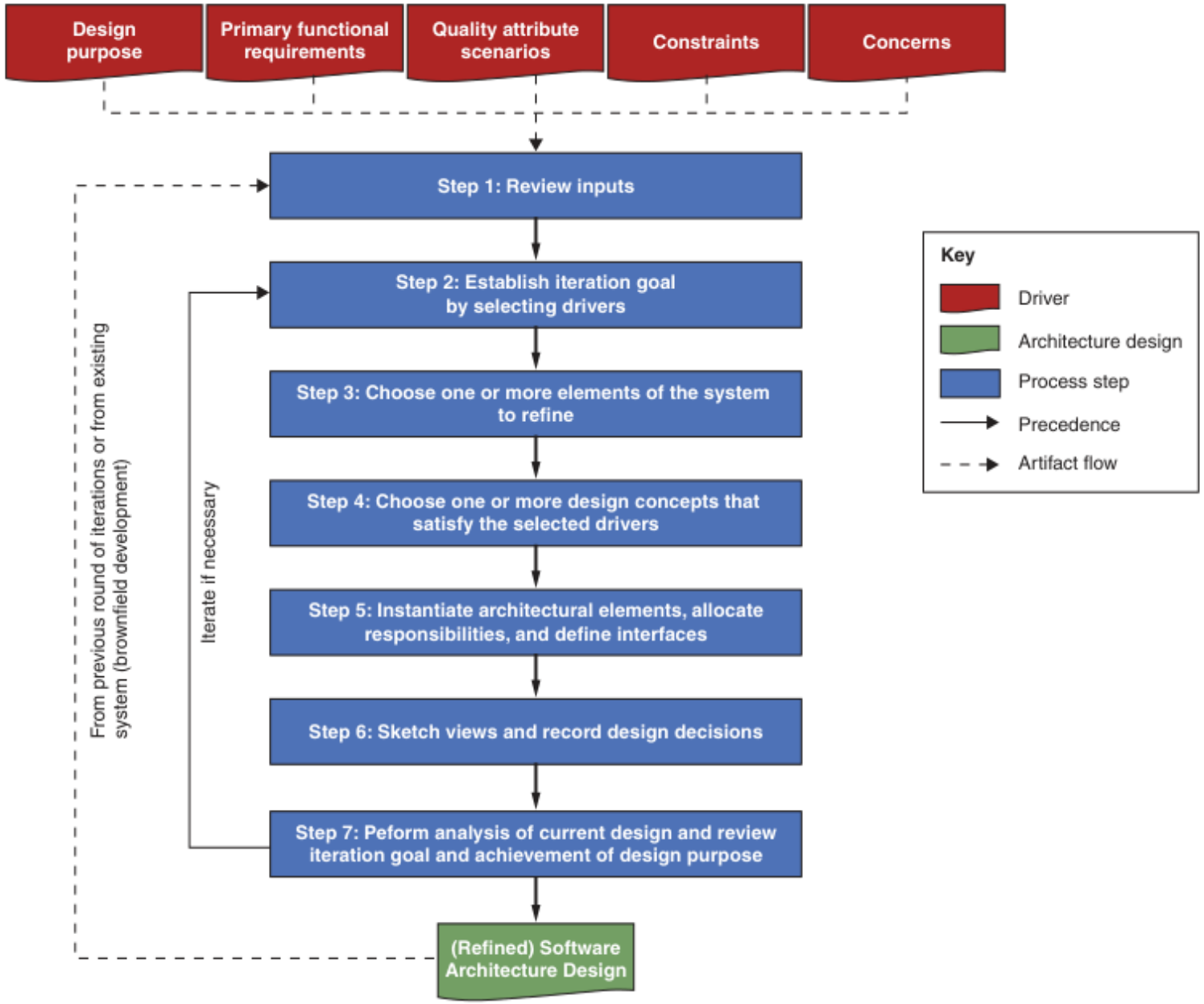


FIGURE 20.3 Steps and artifacts of ADD

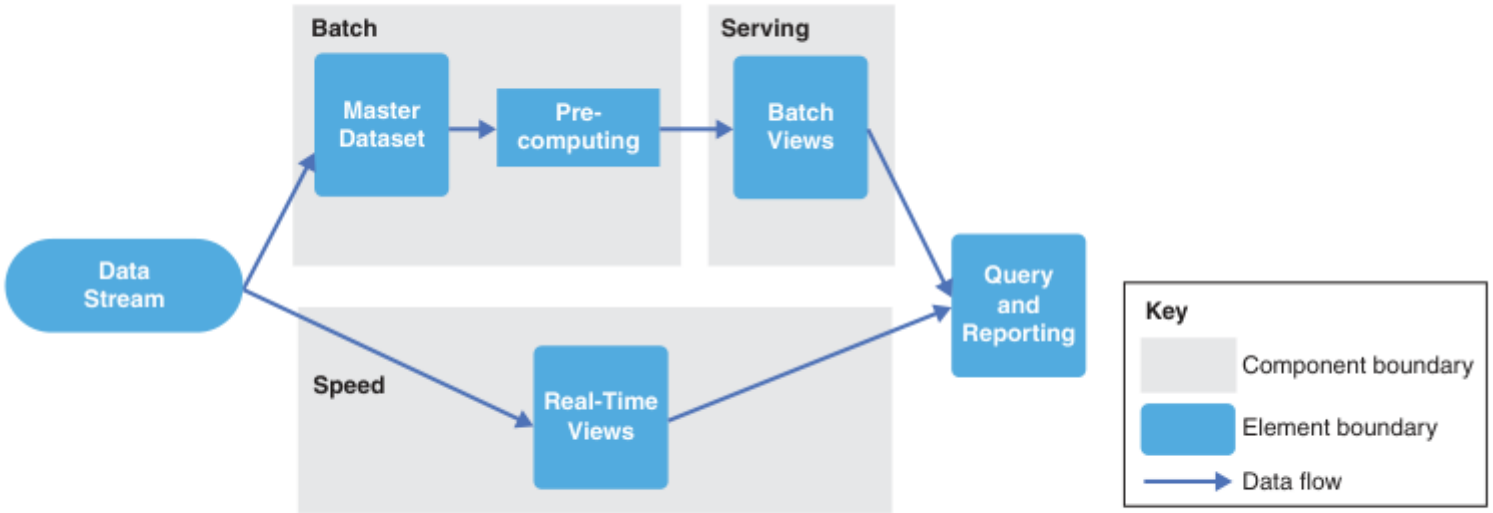


FIGURE 20.4 Example preliminary documentation

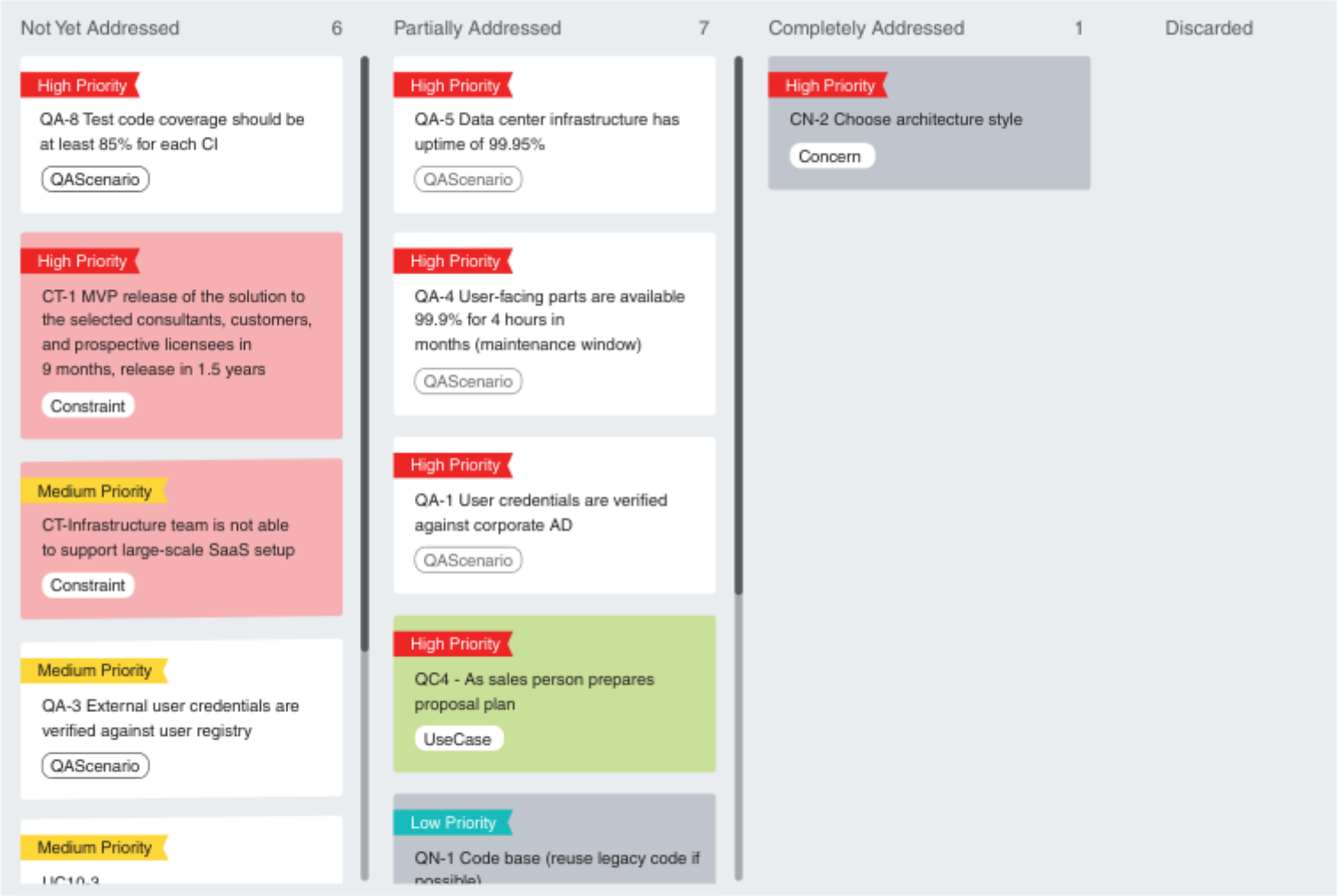


FIGURE 20.5 A Kanban board used to track design progress

Universidad de La Sabana

FACULTAD DE INGENIERÍA

CONTACTO

Francisco Moreno

Profesor Catedra – Facultad de Ingeniería

franciscomodi@unisabana.edu.co

FoSA, Cap 4, 5

SWRe, Cap 14

Universidad de La Sabana
FACULTAD DE INGENIERIA

- Dice modularidad
Primero

Preface
1. Introduction
Defining Software Architecture
Laws of Software Architecture
Expectations of an Architect
Make Architecture Decisions
Continually Analyze the Architecture
Keep Current with Latest Trends
Ensure Compliance with Decisions
Understand Diverse Technologies
Know the Business Domain
Possess Interpersonal Skills
Understand and Navigate Politics
Roadmap
I. Foundations
2. Architectural Thinking
Architecture Versus Design
Strategic Versus Tactical Decisions
Level of Effort
The Significance of Trade-Offs
Technical Breadth
The 20-Minute Rule
Developing a Personal Radar
Analyzing Trade-Offs
Understanding Business Drivers
Balancing Architecture and Hands-On Coding
There's More to Architectural Thinking
3. Modularity
Modularity Versus Granularity
Defining Modularity
Measuring Modularity
From Modules to Components
4. Architectural Characteristics Defined
Architectural Characteristics and System Design
Architectural Characteristics (Partially) Listed
Trade-Offs and Least Worst Architecture

Chapter 3. Modularity

Architects and developers have struggled with the c
quite some time, as is evident in this quote from *Co*
Design (Van Nostrand Reinhold, 1978):

> *95% of the words [written about software archite*
> *extolling the benefits of "modularity" and little,*
> *about how to achieve it.*

—Glenford J

1. Funciones del Arquitecto, FOSA
2. Atributos de Calidad, FOSA y SEI.
3. Tacticas para Disponibilidad (Availability), Rendimiento (Performance) y Despliegue (Deployability). SEI.
4. SLA, SLI, SLO o KPI. SEI y otros.
5. Estilo vs Diseño, FOSA
6. Clasificacion de Estilos: por Despliegue: Monolitos vs Sistemas Distribuidos. SAP Report.
7. Listado de Estilos. FoSA, Azure
8. Listado de Patrones seleccionados. SAP Report, Azure